

Software Engineering

Lecture 7c: White-Box Structural Testing

Prof. Nien-Lin Hsueh

Department of Information Engineering and Computer Science
Feng Chia University

Focus Questions

- What is **White-Box (Structural / Glass-Box) Testing**?
- What is the **Code Coverage Subsumption Hierarchy** (Statement, Branch, Condition, MC/DC, Path)?
- Why does 100% Condition Coverage (CC100) **NOT** guarantee Branch Coverage (BC100)?
- How does **Modified Condition/Decision Coverage (MC/DC)** serve safety-critical software (DO-178B/C)?
- How does **Basis Path Testing** solve the Loop Path Explosion problem?
- Why is the **100% Code Coverage** goal an anti-pattern?

White-Box Structural Testing: Fundamentals

- **White-Box Testing (Structural / Glass-Box Testing):**
 - Evaluates the internal code structure, control flow paths, and implementation logic.
 - **Objective:** Measures how thoroughly the source code is exercised by test cases.
- **Key Coverage Criteria:**
 - **Statement Coverage (SC) → Branch Coverage (BC) → Condition Coverage (CC) → MC/DC → Path Coverage (PC).**

Code Coverage Subsumption Hierarchy

Statement Coverage vs. Branch Coverage

- **Statement Coverage (SC / SC100):**

- *Requirement:* Every executable code statement must be executed at least once.
- *Limitation:* The **weakest criterion**; SC100 can pass even if missing `else` branches or logic operators contain defects.

- **Branch / Decision Coverage (BC / BC100):**

- *Requirement:* Every decision point (`IF`, `ELSE IF`, `WHILE`) must evaluate to **True** at least once and **False** at least once.
- *Subsumption:* Achieving BC100 **guarantees 100% Statement Coverage (SC100)**.

Condition Coverage & The Short-Circuit Paradox

- **Condition Coverage (CC / CC100):**
 - *Requirement:* Every individual atomic boolean condition inside a decision must evaluate to True and False at least once.
- **The CC100 vs. BC100 Paradox:**
 - **CC100 does NOT guarantee BC100!**
 - *Example Decision:* `if (p && q)`
 - Test $t_1 = (\text{True}, \text{False}) \rightarrow \text{False}$
 - Test $t_2 = (\text{False}, \text{True}) \rightarrow \text{False}$
 - Both p and q evaluated to True & False (CC100 achieved), BUT the overall decision `p && q` is ALWAYS False (0% Branch True Coverage!).
- **Short-Circuit Evaluation (`&&`, `||`):**
 - Languages skip evaluating remaining terms if early terms determine the outcome, requiring explicit test cases for unevaluated terms.

Safety-Critical Standard: MC/DC Coverage

- **Modified Condition / Decision Coverage (MC/DC):**
 - Required by **aviation & safety-critical software standards (DO-178B/C)** for flight control, medical devices, and automotive systems.
- **MC/DC Core Requirement:**
 1. Every decision has evaluated to True and False at least once (BC100).
 2. Every condition has evaluated to True and False at least once (CC100).
 3. **Condition Independence:** Toggling a single condition C_i from $T \rightarrow F$ (holding all other conditions constant) **must independently toggle the final decision outcome**.
- **Efficiency:** Requires only **$N + 1$ test cases** for N conditions (vs. 2^N for full Multiple Condition Combination).

Path Coverage & The Loop Explosion Problem

- **Path Coverage (PC):**
 - Tests every independent execution path from program entry to exit.
 - Represents the **most rigorous structural testing criterion**.
- **The Loop Path Explosion Problem:**
 - If a program contains a loop executing 20 times with 4 internal logic paths:
Total Paths = $4^{20} = 2^{40} \approx 1.1 \times 10^{12}$ paths!
 - Full path coverage is computationally impossible for non-trivial programs.
- **Basis Path Testing (McCabe's Cyclomatic Complexity $V(G)$):**
 - Derives a linearly independent set of basis paths to guarantee SC100 and BC100 with $V(G)$ test cases.

The 100% Code Coverage Myth (百分百涵蓋度的迷思)

- **Why 100% Coverage is an Anti-Pattern:**

1. **Diminishing ROI:** Coverage from 0% to 80% yields high quality gains. Pushing from 95% to 100% requires exponential effort for minimal defect reduction.
2. **Execution $\neq$ Correctness:** 100% coverage only proves code was *executed*, not that output assertions are correct, logic is sound (`a - b` vs `a + b`), or missing requirements exist.
3. **Untestable Defensive Code:** Mocking rare OS interrupts or defensive `switch` defaults adds maintenance overhead.

- **Pragmatic Best Practice:** Target 80% -- 95% **coverage** focused on critical core business logic.

Concept Check Question 1

Which statement accurately describes the relationship between Statement Coverage (SC100) and Branch Coverage (BC100)?

- **A.** SC100 guarantees BC100, but BC100 does not guarantee SC100.
- **B.** BC100 guarantees SC100, but SC100 does not guarantee BC100.
- **C.** Neither metric guarantees the other under any circumstances.
- **D.** SC100 and BC100 are mathematically identical metrics.



Concept Check Question 1: Answer

Which statement accurately describes the relationship between Statement Coverage (SC100) and Branch Coverage (BC100)?

- **Correct Answer: B**
- **Explanation:** Executing every branch (BC100) forces every statement within those branches to be executed (SC100), but statement coverage can bypass unexecuted `else` branches.



Recap: Fill-in-the-blank Quiz

Test your understanding of the core concepts in this chapter:

1. coverage requires every decision point to evaluate to True and False at least once.
2. Under short-circuit evaluation, 100% coverage does NOT guarantee branch coverage.
3. The safety-critical standard requires each condition to independently affect the decision outcome using N+1 test cases.
4. McCabe's Complexity $V(G)$ defines the minimum number of basis paths needed for full structural coverage.



Recap: Answers & Summary

Here are the completed concepts:

1. **Branch** coverage requires every decision point to evaluate to True and False at least once.
2. Under short-circuit evaluation, 100% **condition** coverage does NOT guarantee branch coverage.
3. The safety-critical standard **MC/DC** requires each condition to independently affect the decision outcome using N+1 test cases.
4. McCabe's **Cyclomatic** Complexity $V(G)$ defines the minimum number of basis paths needed for full structural coverage.



References

- **Prof. Nien-Lin Hsueh Software Testing Course (Ch06 White-Box Testing)**
 - [FAA / RTCA DO-178C Software Considerations in Airborne Systems](#)
 - [McCabe Cyclomatic Complexity Metric Paper \(IEEE Trans. Softw. Eng. 1976\)](#)